

Perforce Integration — Scope and Opportunity (v3+) Thesis

Vendor access to Perforce is one of the most operationally painful parts of running an outsourced art pipeline: IP exposure, license cost, network setup, depot access management, training overhead, and cleanup when relationships end. ArtHound can sit in front of Perforce as a production-aware broker that gives vendors scoped, audited, revocable access to exactly the depot subsets they need — without ever provisioning them as Perforce users.

This is platform-maturity work, not founding-product work. It amplifies the core product rather than creating it; the value depends on the studio already running vendor relationships through ArtHound.

Architectural pattern

ArtHound holds a Perforce service account. Vendors never have P4 credentials and never touch the depot directly. Reads are scoped pulls from Perforce on the vendor's behalf, bounded by the active payload. Submissions are received by ArtHound, reviewed, and on approval submitted to Perforce as the service account with provenance metadata identifying the original vendor and payload.

Perforce remains the studio's source of truth. Internal artists keep their existing workflow. ArtHound inserts a controlled boundary at the vendor seam. Design surface

Read-side scope. Start explicit (payload lists exact depot paths, ArtHound syncs at snapshot revision); evolve to computed (ArtHound resolves dependencies from the asset graph). Computed is the version that becomes a moat.

Update semantics. Frozen by default — vendor sees the depot state at payload issue. Studio explicitly re-issues payloads to push updates, with each re-issue tracked as a diff against the previous state. Structurally better than Perforce's native "sync to head and figure it out" experience for external collaborators.

Submission round-trip. Single atomic changelist per approved submission. Author identity preserved as structured changelist metadata (P4 user is the service account; real author is the vendor). Conflicts handled by forcing payload re-issue rather than auto-merge — binary asset auto-merge is a non-starter. Exclusive locks taken on payload issue, released on submission or revocation.

Integration layer. P4Python for proper transactional semantics. Connector model for network topology — small studio-side agent talking outbound to ArtHound — rather than requiring VPN access.

Aggressive caching of synced payloads; consider presigned URLs / CDN delivery for large files rather than transiting ArtHound directly. Architectural prep work for v1

The Perforce integration shouldn't be built now, but v1 should not preclude it. Cheap decisions today that prevent expensive refactors later:

Asset model carries a "source reference" field from day one. Today's only value is arthound
Payload dependencies reference assets by ID with a resolution layer between the reference and
Submission flow treats "submit to ArtHound" and "submit to ArtHound, forward to Perforce" as

Strategic positioning

What this describes, when spelled out fully, is a production-aware Perforce broker — a category that doesn't currently exist. Helix Swarm is a UI on top of Perforce for existing P4 users; it doesn't solve vendor access. ShotGrid and ftrack don't broker depot access at all.

Sales pitch sharpens to a multi-stakeholder value prop: studios keep Perforce as source of truth (CTO defensible), vendors get scoped audited access without depot exposure (security and IP), producers get vendor management without depot babysitting (operations), CFOs get fewer Perforce vendor seats (cost). Each stakeholder has their own reason to buy. Sequencing rationale

Build when a specific real studio is pulling on it as a design partner, not speculatively. Generic Perforce support is an unbounded compatibility surface — non-standard server configs, custom triggers, weird depot organization, workflow dependencies on obscure P4 features. The first integration should be built against one studio's actual depot, not against an imagined generic Perforce.